ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 11

Arrays y textos

- Indexados y asociativos
- Arrays dentro de arrays: tablas de datos
- Funciones útiles para arrays
- Ordenar
- array_map y array_filter
- Funciones de texto (y el lío de las tildes)
- Del texto al array y vuelta: explode e implode
- Validar datos: emails y patrones
- Fechas



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Si una variable es una caja donde cabe un solo dato, un **array** es un **archivador con cajones**: cada cajón tiene su número o su etiqueta, y en cada uno guardáis algo. Sin arrays no habría listas de productos, ni carritos de la compra, ni resultados de una base de datos. Son el pan de cada día en PHP.

En la segunda parte de la clase trabajaremos con **textos**: limpiarlos, buscar dentro, cortarlos y comprobar si un email es válido. Y cerraremos con las **fechas**, que también tienen su miga.

1

📘 **Indexados y asociativos**



2

👌 **Arrays dentro de arrays: tablas de datos**



3

👍 **Funciones útiles para arrays**



4

🤔 **Ordenar**



5

🚀 **array_map y array_filter**



6

😮 **Funciones de texto (y el lío de las tildes)**



7

📘 **Del texto al array y vuelta: explode e implode**



8

⚠️ **Validar datos: emails y patrones**



9

🔍 **Fechas**



10

🚀 **Ejercicios**



11

👋 **Lo que tenéis que llevaros de hoy**



🗂️ Arrays



CLASE 11 · PUNTO 1

📘 Indexados y asociativos

En PHP solo hay un tipo de array, pero se usa de dos formas [1]:

	Indexado	Asociativo
Los cajones se identifican por...	Un **número**, que empieza en **0**	Una **etiqueta** (clave) que elegís vosotros
Ejemplo	`$frutas = ["manzana", "pera"];`	`$alumno = ["nombre" ⇒ "Carlos", "nota" ⇒ 8.5];`
Leer un dato	`$frutas[0]` → `manzana`	`$alumno["nombre"]` → `Carlos`
Úsalo para...	Listas de cosas del mismo tipo	Los datos de **una** cosa (una ficha)

Añadir, modificar y eliminar

01-indexados-asociativos.php — php-paso-a-paso/clase11 — Visual Studio Code

01-indexados-asociativos.php

```php
<?php
$frutas[] = "kiwi";                    // añade al FINAL de la lista
$alumno["email"] = "carlos@mail.es";   // añade una etiqueta nueva

$frutas[1] = "melocotón";              // modifica el cajón 1
$alumno["nota"] = 9;                   // modifica la nota

unset($alumno["email"]);               // elimina la etiqueta y su valor
```

Para **ver** todo lo que hay dentro de un array, `print_r()` (más legible) o `var_dump()` (con los tipos). Un truco: envolvedlo en `<pre>` para que el navegador respete los saltos de línea.

01-indexados-asociativos.php — php-paso-a-paso/clase11 — Visual Studio Code

01-indexados-asociativos.php

```php
<?php
echo "<pre>";
print_r($alumno);
echo "</pre>";
```



⚠️ **Los dos errores clásicos:** creer que el primer elemento es el `[1]` (es el `[0]`), y leer una clave que no existe, que da el aviso *Undefined array key*. Para lo segundo ya conocéis la solución: `$alumno["telefono"] ?? "Sin teléfono"`.

📄 Ejemplo: `01-indexados-asociativos.php`



CLASE 11 · PUNTO 2

👌 Arrays dentro de arrays: tablas de datos

En una tienda no tenéis un producto: tenéis una **tabla** de productos, y cada uno con su nombre, precio y stock. Eso es un array de arrays (un array **multidimensional**):

02-multidimensionales.php — php-paso-a-paso/clase11 — Visual Studio Code

02-multidimensionales.php

```php
<?php
$catalogo = [
    ["id" => 1, "nombre" => "Portátil", "precio" => 899.00, "stock" => 3],
    ["id" => 2, "nombre" => "Monitor",  "precio" => 299.50, "stock" => 0],
    ["id" => 3, "nombre" => "Teclado",  "precio" => 49.90,  "stock" => 12],
];

echo $catalogo[0]["nombre"];   // Portátil  → fila 0, columna "nombre"
```

Pensad en él como en una hoja de cálculo: el primer corchete es la **fila** y el segundo, la **columna**.

Y para recorrerlo, `foreach` con la sintaxis alternativa, que es como lo haréis en las páginas de verdad:

02-multidimensionales.php — php-paso-a-paso/clase11 — Visual Studio Code

02-multidimensionales.php

```php
<table>
    <?php foreach ($catalogo as $producto): ?>
        <tr>
            <td><?= $producto["nombre"] ?></td>
            <td><?= $producto["stock"] > 0 ? "En stock" : "Agotado" ?></td>
        </tr>
    <?php endforeach; ?>
</table>
```

📄 Ejemplo: `02-multidimensionales.php`



💡 Fijaos bien en esta estructura: es **exactamente** la forma en que os llegarán los datos de una base de datos MySQL. Lo que practiquéis hoy lo usaréis tal cual más adelante.



CLASE 11 · PUNTO 3

👍 Funciones útiles para arrays

PHP trae decenas de funciones para arrays [2]. Estas son las que vais a usar a diario:

Función	Qué hace	Ejemplo	Resultado
`count()`	Cuántos elementos hay	`count([7, 4, 9])`	`3`
`array_sum()`	Suma todos los valores	`array_sum([7, 4, 9])`	`20`
`max()` / `min()`	El mayor / el menor	`max([7, 4, 9])`	`9`
`in_array()`	¿Existe este **valor**?	`in_array("admin", $roles)`	`true` / `false`
`array_key_exists()`	¿Existe esta **clave**?	`array_key_exists("email", $usuario)`	`true` / `false`
`array_search()`	¿En qué posición está el valor?	`array_search(9, [7, 4, 9])`	`2`
`array_merge()`	Une dos arrays	`array_merge(["pan"], ["leche"])`	`["pan", "leche"]`
`array_keys()`	Solo las claves	`array_keys($usuario)`	`["nombre", "email"]`
`array_unique()`	Quita los repetidos	`array_unique([1, 1, 2])`	`[1, 2]`

📄 Ejemplo: `03-funciones-arrays.php`



CLASE 11 · PUNTO 4

🤔 Ordenar

Función	Ordena por...	Sentido	¿Conserva las claves?
`sort()`	Valor	Menor a mayor	❌ No: renumera desde 0
`rsort()`	Valor	Mayor a menor	❌ No
`asort()`	Valor	Menor a mayor	✅ Sí
`arsort()`	Valor	Mayor a menor	✅ Sí
`ksort()`	Clave	A → Z	✅ Sí

La regla es sencilla: con arrays **indexados**, `sort()` y `rsort()`. Con arrays **asociativos**, las que empiezan por `a` o por `k`, porque si usáis `sort()` perdéis las etiquetas [3].

04-ordenar.php — php-paso-a-paso/clase11 — Visual Studio Code

04-ordenar.php

```php
<?php
$notas = ["Marta" => 8, "Luis" => 6, "Ana" => 9];

arsort($notas);    // Ana 9, Marta 8, Luis 6  → un ranking
ksort($notas);     // Ana, Luis, Marta        → orden alfabético
```



⚠️ Estas funciones **modifican el array original** y no devuelven el array ordenado. `$ordenado = sort($notas);` no hace lo que creéis: guarda `true` en `$ordenado`.

📄 Ejemplo: `04-ordenar.php`



CLASE 11 · PUNTO 5

🚀 array_map y array_filter

¿Os acordáis de las funciones flecha de la clase 10? Aquí es donde brillan.

Función	Qué hace	Analogía
`array_map()`	**Transforma** cada elemento y devuelve un array nuevo	Pasar todas las prendas por la plancha
`array_filter()`	Se **queda** con los elementos que cumplen una condición	Pasar las lentejas por el colador



Fijaos en un detalle tonto que siempre confunde: en `array_map()` la función va **primero** y el array después; en `array_filter()`, al revés.

¿Es obligatorio usarlas? No. Todo lo que hacen se puede hacer con un `foreach`, y en el ejemplo tenéis las dos versiones. Pero os las encontraréis en todas partes, sobre todo en Laravel, así que conviene saber leerlas.

📄 Ejemplo: `05-map-filter.php`

🔤 Textos y fechas



CLASE 11 · PUNTO 6

😮 Funciones de texto (y el lío de las tildes)

El lío de las tildes

06-textos.php — php-paso-a-paso/clase11 — Visual Studio Code

06-textos.php

```php
<?php
echo strlen("Ramón");      // 6  ← ¿6? ¡Si tiene 5 letras!
echo mb_strlen("Ramón");   // 5  ← ahora sí
```

¿Qué ha pasado? Las funciones clásicas de texto cuentan **bytes**, no letras, y en UTF-8 la «ó» ocupa dos bytes. Con las **tildes, las eñes o los emojis**, fallan. Por eso existen las versiones `mb_` (*multibyte*), que cuentan letras de verdad [4].



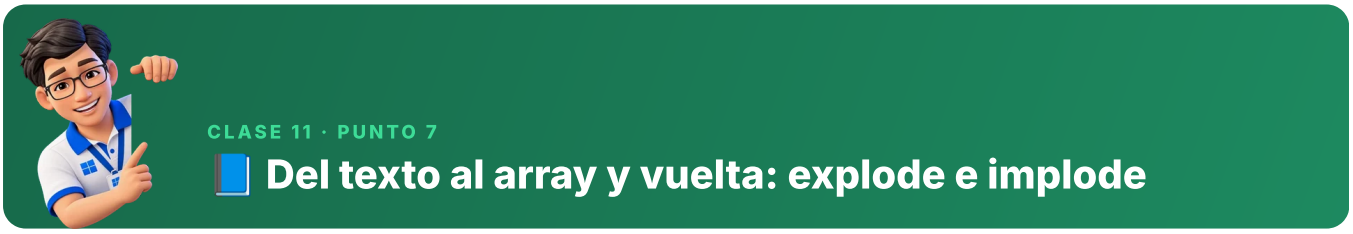
🎯 **Regla para textos en español:** si existe la versión `mb_` de una función, usad la versión `mb_`.

Las que vais a usar a diario

Función	Qué hace	Ejemplo	Resultado
`trim()`	Quita los espacios de los extremos	`trim(" hola ")`	`"hola"`
`mb_strlen()`	Longitud en letras	`mb_strlen("Ramón")`	`5`
`mb_strtoupper()` / `mb_strtolower()`	Mayúsculas / minúsculas	`mb_strtoupper("ramón")`	`"RAMÓN"`
`mb_convert_case(..., MB_CASE_TITLE)`	Primera letra de cada palabra en mayúscula	`"ramón abramo"`	`"Ramón Abramo"`
`mb_substr()`	Un trozo del texto (desde, cuántas)	`mb_substr("Programación", 0, 7)`	`"Program"`
`str_replace()`	Reemplaza un texto por otro	`str_replace("mundo", "clase", "Hola, mundo")`	`"Hola, clase"`
`str_contains()`	¿Contiene...?	`str_contains($email, "@")`	`true`
`str_starts_with()` / `str_ends_with()`	¿Empieza / termina por...?	`str_ends_with("foto.jpg", ".pdf")`	`false`

Las tres últimas llegaron con PHP 8 y son una maravilla: se leen como una frase [5].

📄 Ejemplo: `06-textos.php`



Dos funciones que hacen de puente:

- `explode()` **parte** un texto en trozos y os da un array.
- `implode()` **une** los elementos de un array en un texto.

07-explode-implode.php — php-paso-a-paso/clase11 — Visual Studio Code

07-explode-implode.php

```php
<?php
$etiquetas = "php, laragon, mysql, apache";
$lista = explode(", ", $etiquetas);   // ["php", "laragon", "mysql", "apache"]

echo implode(" - ", $lista);          // php - laragon - mysql - apache
```

El primer parámetro es el **separador**: por dónde cortar o con qué pegar. Un caso muy real: las líneas de un archivo CSV, separadas por `;`.

07-explode-implode.php — php-paso-a-paso/clase11 — Visual Studio Code

07-explode-implode.php

```php
<?php
$datos = explode(";", "Marta;García;22;Santander");
echo $datos[3];   // Santander
```

📄 Ejemplo: `07-explode-implode.php`



CLASE 11 · PUNTO 8

⚠️ Validar datos: emails y patrones

Emails: no inventéis nada

Comprobar a mano si un email es correcto es dificilísimo. PHP ya lo trae resuelto con `filter_var()` [6]:

08-validar.php — php-paso-a-paso/clase11 — Visual Studio Code

08-validar.php

```php
<?php
if (filter_var($email, FILTER_VALIDATE_EMAIL) == false) {
    echo "Email válido";
}
```

Si es válido, devuelve el email; si no, devuelve `false`. Por eso comparamos con `== false`.

Para saber más: expresiones regulares

Una **expresión regular** es un patrón que describe cómo tiene que ser un texto. Son muy potentes, pero también muy crípticas, así que hoy solo os enseño una para que os suenen cuando las veáis [7]:

08-validar.php — php-paso-a-paso/clase11 — Visual Studio Code

08-validar.php

```php
<?php
// Teléfono español: empieza por 6, 7, 8 o 9 y le siguen 8 cifras
if (preg_match('/^[6-9][0-9]{8}$/', $telefono) == 1) {
    echo "Teléfono válido";
}
```

Trozo	Significa
`^`	Aquí empieza el texto
`[6-9]`	Un carácter entre el 6 y el 9
`[0-9]{8}`	Ocho cifras del 0 al 9
`$`	Aquí termina el texto

No hace falta que las sepáis escribir ahora. Si algún día necesitáis una, la construiréis con calma y con la documentación delante.

📄 Ejemplo: `08-validar.php`



date(): mostrar la fecha

09-fechas.php — php-paso-a-paso/clase11 — Visual Studio Code

09-fechas.php

```php
<?php
date_default_timezone_set("Europe/Madrid");

echo date("d/m/Y");        // 06/11/2026
echo date("d/m/Y H:i");    // 06/11/2026 17:45
```

Letra	Significa	Ejemplo
`d` / `m` / `Y`	Día / mes / año con 4 cifras	`06` / `11` / `2026`
`H` / `i` / `s`	Hora (0-23) / minutos / segundos	`17` / `45` / `03`



⚠️ Si la hora os sale con una o dos horas de diferencia, es la **zona horaria**. Indicadla al principio con `date_default_timezone_set("Europe/Madrid");` [8].

DateTime: hacer cuentas con fechas

Para calcular una edad, sumar días o saber cuánto falta para algo, PHP tiene `DateTime` [9]. Es un **objeto**, y esa flecha `→` que vais a ver es la forma de usarlo. No os preocupéis por ella ahora: los objetos son el plato fuerte de la segunda parte del curso.

09-fechas.php — php-paso-a-paso/clase11 — Visual Studio Code

09-fechas.php

```php
<?php
$hoy = new DateTime();
$nacimiento = new DateTime("2000-05-20");
echo "Edad: " . $nacimiento->diff($hoy)->y . " años";

$entrega = new DateTime("2026-11-06");
$entrega->modify("+15 days");
echo $entrega->format("d/m/Y");    // 21/11/2026
```

📄 Ejemplo: `09-fechas.php`



CLASE 11 · PUNTO 10

🚀 Ejercicios

- ☐ **01 · Agenda.** Un array de contactos (nombre, teléfono y ciudad). Añadid uno nuevo, mostradlos en una lista `<ul>` y contad cuántos son de Santander con `array_filter()`. → `ej01-agenda.php`
- ☐ **02 · Ranking.** Con `["Marta" ⇒ 8.5, "Luis" ⇒ 4.2, ...]`, mostrad un ranking de mayor a menor nota en una lista `<ol>` y la media. → `ej02-ranking.php`
- ☐ **03 · Nombre de usuario.** `crearUsuario(string $nombre, string $apellido): string`: inicial del nombre más apellido, en minúsculas y sin espacios. «Ramón» y «Abramo» → `rabramo`. → `ej03-nombre-usuario.php`
- ☐ **04 · Etiquetas limpias.** A partir de `"PHP, html , css,php , MySQL"`, obtened `css · html · mysql · php`: sin espacios, en minúsculas, sin repetidos y en orden. → `ej04-etiquetas.php`
- ☐ **05 · Cuenta atrás.** ¿Cuántos días faltan para el 22 de diciembre? Si ya ha pasado este año, contad hasta el del año que viene. → `ej05-dias-vacaciones.php`



🏆 **Reto:** en el ejercicio 03 probad con «Lucía Fernández». El usuario sale con tilde: `lfer-nández`. ¿Cómo lo quitaríais? Pista: `str_replace()` admite arrays como primer y segundo parámetro.



CLASE 11 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Indexado	Posiciones numéricas desde el **0**: listas
Asociativo	Claves con nombre: fichas de datos
Multidimensional	Una tabla: `$catalogo[fila]["columna"]`
Ver un array	`print_r()` dentro de `<pre>`
Ordenar	`sort()` para listas; `asort()`, `arsort()` y `ksort()` para asociativos
`array_map` / `array_filter`	Transformar todos / quedarse con algunos
Tildes	Usad las funciones `mb_`
`explode` / `implode`	Texto → array / array → texto
Emails	`filter_var($email, FILTER_VALIDATE_EMAIL)`
Fechas	`date()` para mostrar; `DateTime` para calcular

📚 Fuentes y documentación oficial

1. Manual de PHP: arrays
2. Manual de PHP: funciones de arrays
3. Manual de PHP: ordenación de arrays
4. Manual de PHP: cadenas de caracteres multibyte (mbstring)
5. Manual de PHP: funciones de strings
6. Manual de PHP: filter_var
7. Manual de PHP: preg_match
8. Manual de PHP: date_default_timezone_set
9. Manual de PHP: la clase DateTime